

CAREERSENSE ACADEMY

PROFESSIONAL LEARNING HANDBOOK

SQL for Professionals

From reliable queries to analytical thinking, reporting, optimization, and
real-world business decisions

For analysts • engineers • product professionals • consultants • career switchers

CareerSense Professional Series • careersenseai.com/resources

How to Use This Handbook

This book is designed for professionals who already know that SQL is more than syntax. The real skill is turning ambiguous business questions into trustworthy queries, checking assumptions, explaining results, and writing SQL that another person can safely maintain.

What each chapter contains

Each chapter follows the same practical rhythm: business context, learning objectives, concepts, worked SQL, interpretation, common mistakes, a professional habit, revision notes, practice questions, and hands-on exercises.

PROFESSIONAL FOCUS

Examples use realistic analytics situations such as customers, orders, subscriptions, interviews, candidates, and job applications. The goal is not just to make a query run - it is to make the result useful, explainable, and defensible.

How to practise

- Type every query yourself instead of only reading it.
- Predict the result before you run the query.
- Check row counts before and after joins.
- Explain your result in one sentence of business language.
- Rewrite working queries for clarity after they are correct.

CONTENTS

What You Will Learn

- | | |
|---|--|
| 1 | Thinking in Tables: SELECT, WHERE, ORDER BY |
| 2 | Aggregations That Answer Business Questions |
| 3 | Joins Without Accidental Duplication |
| 4 | CASE, NULLs, Dates, and Data Quality |
| 5 | CTEs and Subqueries for Readable Analysis |
| 6 | Window Functions for Professional Analytics |
| 7 | Query Performance, Index Thinking, and Explainability |
| 8 | CareerSense Mini Project: Job Search Analytics Dashboard |

Closing the book: Whole-book revision • Glossary • 20-question professional quiz • Answer key

01

Thinking in Tables: SELECT, WHERE, ORDER BY

A professional SQL query starts before SELECT. It starts with a precise question: what is the grain of the result, which rows belong, and what should one row mean when the query is finished?

LEARNING OBJECTIVES

- Translate a business question into a table grain
- Select only useful columns
- Filter rows safely with WHERE
- Sort results intentionally
- Use aliases to make results readable
- Spot common filtering mistakes

Main Concepts

The core SELECT workflow is deceptively simple: choose columns, choose a source, filter rows, and optionally sort the result. Professional work adds one more step: define what a row represents. If one row should mean one active customer, your query should not silently return one row per order.

PROFESSIONAL HABIT

Before writing SQL, complete this sentence: “One row in my final result represents ____.” This single sentence prevents many downstream mistakes.

Worked Example - active enterprise customers

SQL

```
SELECT
  customer_id,
  company_name,
  country,
  annual_contract_value AS acv
FROM customers
WHERE status = 'active'
AND segment = 'enterprise'
```

```
ORDER BY annual_contract_value DESC;
```

RESULT

Returns active enterprise customers, highest ACV first.

Reading it professionally

The WHERE clause defines eligibility, while ORDER BY only changes presentation. The alias acv shortens the output without changing the stored column. A reviewer can understand the intent without knowing the schema in advance.

Filtering patterns worth knowing

SQL

```
SELECT application_id, candidate_id, applied_at
FROM job_applications
WHERE applied_at >= DATE '2026-01-01'
  AND status IN ('screening', 'interview')
  AND referral_source IS NOT NULL
ORDER BY applied_at DESC;
```

COMMON MISTAKE

Using = NULL or <> NULL. NULL is not a normal value; use IS NULL or IS NOT NULL.

QUICK REVISION NOTES

- Define the grain before the query.
- SELECT controls columns; WHERE controls rows.
- ORDER BY affects display order, not eligibility.
- Use IN for compact multi-value filters.
- Use IS NULL / IS NOT NULL for missing values.

Practice Questions

- Q1** What does “grain” mean in a SQL result?
- Q2** What is the difference between WHERE and ORDER BY?
- Q3** Why is = NULL incorrect?
- Q4** When would IN be clearer than multiple OR conditions?

Hands-on Exercises

- 1 Write a query for active customers in India ordered by signup date.
- 2 Return interviews scheduled in the next 30 days.

3 Rewrite a query that uses `SELECT *` so it returns only business-relevant columns.

02

Aggregations That Answer Business Questions

A count is only meaningful when you know what is being counted. Aggregations turn detail rows into measures, but GROUP BY determines the level at which those measures are reported.

LEARNING OBJECTIVES

- Use COUNT, SUM, AVG, MIN, and MAX
- Group results at the correct business level
- Distinguish COUNT(*) from COUNT(column)
- Filter grouped results with HAVING
- Avoid misleading averages
- Calculate ratios safely

The aggregation question

Every aggregation has two parts: the measure and the grouping level. “Revenue by month” means SUM(revenue) as the measure and month as the grouping level. If you group by too many columns, the result becomes fragmented; if you group by too few, useful distinctions disappear.

SQL

```
SELECT
  DATE_TRUNC('month', order_date) AS month,
  COUNT(*) AS orders,
  SUM(order_amount) AS revenue,
  AVG(order_amount) AS avg_order_value
FROM orders
WHERE order_status = 'completed'
GROUP BY DATE_TRUNC('month', order_date)
ORDER BY month;
```

BUSINESS INTERPRETATION

This result supports three different statements: how many completed orders occurred, how much revenue they generated, and how large the average completed order was. Do not collapse these into one vague “performance” metric.

HAVING vs WHERE

SQL

```
SELECT customer_id, SUM(order_amount) AS lifetime_revenue
FROM orders
WHERE order_status = 'completed'
GROUP BY customer_id
HAVING SUM(order_amount) >= 100000
ORDER BY lifetime_revenue DESC;
```

WHERE filters detail rows before aggregation. HAVING filters the grouped result after aggregation. That sequencing is why HAVING can reference SUM(order_amount).

COMMON MISTAKE

Using AVG on a skewed distribution and presenting it as “typical.” For salaries, deal sizes, or response times, compare average with median when your database supports it or inspect percentiles.

QUICK REVISION NOTES

- Aggregation changes the grain.
- GROUP BY defines the reporting level.
- COUNT(*) counts rows; COUNT(column) ignores NULLs in that column.
- WHERE filters before grouping; HAVING filters after grouping.
- Always name the numerator and denominator of a ratio.

Practice Questions

- Q1** What two decisions define an aggregation?
- Q2** How does COUNT(*) differ from COUNT(email)?
- Q3** When should HAVING be used?
- Q4** Why can averages be misleading?

Hands-on Exercises

- 1 Calculate interviews per recruiter.
- 2 Find customers with more than five completed orders.
- 3 Calculate monthly application-to-interview conversion rate.

03

Joins Without Accidental Duplication

Joins are where technically correct SQL can produce professionally wrong results. The key question is not “which join works?” but “what relationship exists between these tables, and how many rows should that relationship create?”

LEARNING OBJECTIVES

- Explain one-to-one, one-to-many, and many-to-many relationships
- Use INNER and LEFT JOIN intentionally
- Predict row multiplication before joining
- Diagnose duplicate-looking results
- Join on complete business keys
- Validate joins with row counts

Relationships come first

If one customer has many orders, joining customers to orders changes the grain from one row per customer to one row per order. That is expected. Problems begin when you still interpret the result as customer-level data.

SQL

```
SELECT
  c.customer_id,
  c.company_name,
  o.order_id,
  o.order_amount
FROM customers c
JOIN orders o
  ON o.customer_id = c.customer_id
WHERE o.order_status = 'completed';
```

VALIDATION HABIT

Before and after a join, check COUNT(*), COUNT(DISTINCT primary_key), and a few known entities. If the distinct entity count changes unexpectedly, stop and investigate.

LEFT JOIN for coverage questions

SQL

```
SELECT
  c.customer_id,
  c.company_name,
  COUNT(o.order_id) AS completed_orders
FROM customers c
LEFT JOIN orders o
  ON o.customer_id = c.customer_id
 AND o.order_status = 'completed'
GROUP BY c.customer_id, c.company_name
ORDER BY completed_orders DESC;
```

Putting the order-status filter in the ON clause preserves customers with zero completed orders. Moving the same filter to WHERE would remove the NULL-extended rows and effectively turn the LEFT JOIN into an inner join for this condition.

COMMON MISTAKE

Joining two fact-like tables directly, such as orders and support_tickets, only by customer_id. If a customer has 5 orders and 4 tickets, the join can create 20 rows. Aggregate each side first or join through the correct dimensional grain.

QUICK REVISION NOTES

- Know the relationship before choosing the join.
- A one-to-many join changes the grain.
- LEFT JOIN preserves unmatched left-side rows.
- Filter placement can change LEFT JOIN behavior.
- Validate joins with distinct key counts.

Practice Questions

- Q1** Why can a join create more rows than either input table?
- Q2** What happens when a right-table filter is placed in WHERE after a LEFT JOIN?
- Q3** What metrics would you check to validate a join?
- Q4** When should two fact tables be aggregated before joining?

Hands-on Exercises

- 1 Find customers with zero orders.
- 2 Join applications to interviews without losing applications that never reached interview.
- 3 Create a one-row-per-customer revenue table before joining it to customer attributes.

04

CASE, NULLs, Dates, and Data Quality

Professional SQL often lives in imperfect data. CASE expressions, careful NULL handling, and explicit date logic let you turn messy source fields into stable analytical categories without hiding uncertainty.

LEARNING OBJECTIVES

- Create business categories with CASE
- Handle NULL values consciously
- Use COALESCE for presentation and fallback logic
- Work with date differences and periods
- Separate unknown from zero
- Build simple data-quality checks

CASE as business logic

SQL

```
SELECT
  candidate_id,
  total_experience_years,
  CASE
    WHEN total_experience_years < 2 THEN 'early career'
    WHEN total_experience_years < 6 THEN 'mid career'
    WHEN total_experience_years < 12 THEN 'senior'
    ELSE 'leadership'
  END AS career_stage
FROM candidates;
```

CASE evaluates conditions from top to bottom. The order matters: once a condition matches, later branches are skipped.

NULL is unknown, not zero

SQL

```
SELECT
```

```
application_id,
COALESCE(referral_source, 'not recorded') AS referral_source
FROM job_applications;
```

DATA-QUALITY NOTE

COALESCE is useful for reporting, but replacing NULL with 0 can change meaning. “Salary not recorded” is not the same as “salary is zero.” Preserve semantic differences where they matter.

Date-based metrics

SQL

```
SELECT
  application_id,
  applied_at,
  first_interview_at,
  DATE_DIFF('day', applied_at, first_interview_at) AS days_to_first_interview
FROM application_funnel
WHERE first_interview_at IS NOT NULL;
```

Date functions vary across databases. The analytical idea is stable: define the two events, choose a unit, and ensure both timestamps represent comparable moments.

COMMON MISTAKE

Treating missing dates as evidence that an event never happened. A NULL interview date can mean no interview, or it can mean incomplete data. Check the source process before interpreting it.

QUICK REVISION NOTES

- CASE applies ordered business rules.
- NULL represents missing or unknown information.
- COALESCE provides fallback values but can hide meaning if misused.
- Define date metrics using explicit start and end events.
- Data quality is part of analysis, not a separate afterthought.

Practice Questions

- Q1** Why does CASE order matter?
- Q2** What is the semantic risk of replacing NULL salary with 0?
- Q3** What does COALESCE do?
- Q4** What should be defined before calculating a time-to-event metric?

Hands-on Exercises

- 1 Create salary bands with CASE.
- 2 Label missing company size as “unknown” without changing real zeros.
- 3 Calculate days from application to offer for completed funnels.

05

CTEs and Subqueries for Readable Analysis

Complex SQL becomes professional when another analyst can inspect it, test its pieces, and explain its logic. Common table expressions let you name intermediate steps and make reasoning visible.

LEARNING OBJECTIVES

- Use CTEs to break analysis into steps
- Choose between CTEs and subqueries
- Build multi-stage metrics safely
- Reduce duplicated logic
- Name intermediate datasets clearly
- Debug one stage at a time

A CTE is a named step

SQL

```
WITH customer_revenue AS (  
  SELECT  
    customer_id,  
    SUM(order_amount) AS revenue  
  FROM orders  
  WHERE order_status = 'completed'  
  GROUP BY customer_id  
)  
customer_segments AS (  
  SELECT  
    customer_id,  
    CASE  
      WHEN revenue >= 100000 THEN 'strategic'  
      WHEN revenue >= 25000 THEN 'growth'  
      ELSE 'standard'  
    END AS value_segment  
  FROM customer_revenue  
)  
SELECT *
```

```
FROM customer_segments
ORDER BY customer_id;
```

The query is longer than an equivalent nested expression, but the reasoning is clearer: first calculate revenue, then classify it. Each CTE has a single job.

PROFESSIONAL HABIT

Name CTEs after what the rows represent, not the operation used to build them. `customer_revenue` is clearer than `cte1` or `grouped_data`.

When a subquery is enough

SQL

```
SELECT *
FROM (
  SELECT customer_id, SUM(order_amount) AS revenue
  FROM orders
  GROUP BY customer_id
) x
WHERE revenue >= 50000;
```

A short one-off subquery can be perfectly readable. Prefer a CTE when there are multiple stages, repeated references, or a need to test intermediate results.

QUICK REVISION NOTES

- CTEs expose the stages of reasoning.
- Each intermediate step should have a clear grain.
- Readable SQL is easier to validate and maintain.
- Subqueries are fine for compact local logic.
- Do not repeat the same business rule in multiple places if one named step can define it once.

Practice Questions

- Q1** What is the main professional benefit of a CTE?
- Q2** When might a subquery be clearer than a CTE?
- Q3** How should CTEs be named?
- Q4** Why is repeated business logic risky?

Hands-on Exercises

- 1 Build a CTE that calculates monthly revenue, then a second that calculates month-over-month status labels.
- 2 Rewrite a deeply nested query using named CTEs.

3 Debug a three-stage CTE by running each stage independently.

06

Window Functions for Professional Analytics

Window functions are one of the biggest steps from intermediate SQL to professional analytics. They let you compare rows, rank entities, and calculate running metrics without collapsing detail rows.

LEARNING OBJECTIVES

- Understand PARTITION BY and ORDER BY inside windows
- Use ROW_NUMBER and RANK
- Calculate running totals
- Use LAG and LEAD for comparisons
- Find latest records per entity
- Separate window logic from GROUP BY logic

Windows preserve detail

GROUP BY reduces many rows into fewer rows. Window functions calculate across related rows while preserving the original detail. This makes them ideal for rankings, trends, and “latest record” problems.

SQL

```
SELECT
  customer_id,
  order_id,
  order_date,
  order_amount,
  ROW_NUMBER() OVER (
    PARTITION BY customer_id
    ORDER BY order_date DESC
  ) AS order_recency_rank
FROM orders;
```

USE CASE

Filter `order_recency_rank = 1` in an outer query to get the latest order per customer. This is usually safer than joining on `MAX(order_date)` when multiple orders can share the same date.

Running totals

SQL

```
SELECT
  order_date,
  order_amount,
  SUM(order_amount) OVER (
    ORDER BY order_date
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS cumulative_revenue
FROM orders
WHERE order_status = 'completed';
```

Period-over-period comparison

SQL

```
WITH monthly AS (
  SELECT
    DATE_TRUNC('month', order_date) AS month,
    SUM(order_amount) AS revenue
  FROM orders
  GROUP BY DATE_TRUNC('month', order_date)
)
SELECT
  month,
  revenue,
  LAG(revenue) OVER (ORDER BY month) AS prior_month_revenue,
  revenue - LAG(revenue) OVER (ORDER BY month) AS change_vs_prior
FROM monthly;
```

COMMON MISTAKE

Using ROW_NUMBER without a deterministic ORDER BY. If ties are possible and you do not specify how to break them, the selected “first” row can change between runs.

QUICK REVISION NOTES

- Window functions preserve rows.
- PARTITION BY defines peer groups.
- ORDER BY inside OVER defines sequence.
- ROW_NUMBER is useful for latest-record logic.
- LAG and LEAD compare neighboring periods or events.

Practice Questions

- Q1** How do window functions differ from GROUP BY?
- Q2** What does PARTITION BY do?
- Q3** When is ROW_NUMBER useful?
- Q4** What problem does LAG solve?

Hands-on Exercises

- 1** Find each customer's latest completed order.
- 2** Rank candidates by assessment score within each target role.
- 3** Calculate monthly revenue and prior-month revenue.

07

Query Performance, Index Thinking, and Explainability

Professional SQL has two audiences: the database engine and the people who maintain the analysis. Performance matters, but so do clarity, reproducibility, and the ability to explain why the query returns what it returns.

LEARNING OBJECTIVES

- Recognize common performance anti-patterns
- Understand why selective filters help
- Think about indexes conceptually
- Avoid unnecessary columns and repeated scans
- Use EXPLAIN conceptually
- Balance optimization with readability

Start with less work

- Select only the columns you need.
- Filter early when it does not change the intended logic.
- Avoid joining tables that contribute no required fields or filters.
- Aggregate before joining when detail is unnecessary.
- Avoid wrapping indexed filter columns in unnecessary functions when your database cannot use the index efficiently.

SQL

```
-- Less efficient pattern
SELECT *
FROM orders
WHERE EXTRACT(YEAR FROM order_date) = 2026;

-- Often more index-friendly
SELECT order_id, customer_id, order_amount
FROM orders
WHERE order_date >= DATE '2026-01-01'
AND order_date < DATE '2027-01-01';
```

INDEX THINKING

An index is not “make query fast” magic. Conceptually, it gives the database a faster path to particular values. It helps most when filters or joins are selective and align with indexed columns. Exact behavior depends on the database engine.

EXPLAIN as a conversation with the optimizer

Most databases provide an EXPLAIN or query-plan tool. You do not need to memorize every operator to benefit from it. Start by asking: Is the engine scanning an entire large table? Which joins are expensive? Are row estimates wildly different from actual results? Is the query repeatedly scanning the same data?

COMMON MISTAKE

Optimizing a query before confirming the business logic. A fast wrong query is worse than a slow correct one. First validate the result, then improve performance while protecting semantics.

QUICK REVISION NOTES

- Reduce unnecessary work first.
- Filtering and join keys strongly influence performance.
- Indexes help access paths but depend on engine and workload.
- Use query plans to investigate, not guess.
- Correctness and maintainability come before micro-optimization.

Practice Questions

- Q1** Why can SELECT * hurt performance and maintainability?
- Q2** What is the conceptual purpose of an index?
- Q3** What should you look for in a query plan?
- Q4** Why should optimization follow correctness validation?

Hands-on Exercises

- 1 Rewrite a year-extraction filter as a date range.
- 2 Remove unused joins and columns from an existing query.
- 3 Compare the plan of a broad query and a selective query in your SQL environment.

08

CareerSense Mini Project: Job Search Analytics Dashboard

The best way to consolidate professional SQL is to build one analysis from raw activity to decision-ready metrics. This mini project models a job-search funnel and asks you to create a compact dashboard dataset.

LEARNING OBJECTIVES

- Design a useful analytical grain
- Build funnel metrics from multiple tables
- Use joins, CTEs, CASE, and windows together
- Create conversion and time-to-event measures
- Check data quality before presenting results
- Write a short business interpretation

Scenario

CareerSense wants a dashboard that helps a job seeker understand whether their search is improving. The source tables are applications, interviews, offers, and companies. The final result should contain one row per calendar month.

Target metrics

- Applications submitted
- Applications reaching interview
- Offers received
- Application-to-interview conversion rate
- Interview-to-offer conversion rate
- Average days to first interview
- Top source of applications
- Month-over-month change in applications

Suggested build

1. Create an `application_base` CTE with one row per application.
2. Aggregate interview events to one row per application with `first_interview_at`.

3. Aggregate offers to one row per application.
4. Join those one-row-per-application datasets.
5. Create monthly funnel metrics.
6. Use LAG to calculate month-over-month change.
7. Run quality checks on distinct application counts before publishing.

SQL

```
WITH first_interview AS (  
  SELECT  
    application_id,  
    MIN(interview_at) AS first_interview_at  
  FROM interviews  
  GROUP BY application_id  
,  
offer_flag AS (  
  SELECT  
    application_id,  
    MIN(offer_at) AS first_offer_at  
  FROM offers  
  GROUP BY application_id  
,  
funnel AS (  
  SELECT  
    a.application_id,  
    a.applied_at,  
    a.source,  
    fi.first_interview_at,  
    ofr.first_offer_at  
  FROM applications a  
  LEFT JOIN first_interview fi  
    ON fi.application_id = a.application_id  
  LEFT JOIN offer_flag ofr  
    ON ofr.application_id = a.application_id  
,  
monthly AS (  
  SELECT  
    DATE_TRUNC('month', applied_at) AS month,  
    COUNT(*) AS applications,  
    COUNT(first_interview_at) AS interviewed,  
    COUNT(first_offer_at) AS offers  
  FROM funnel  
  GROUP BY DATE_TRUNC('month', applied_at)  
)  
SELECT  
  month,  
  applications,  
  interviewed,  
  offers,
```

```

1.0 * interviewed / NULLIF(applications, 0) AS interview_rate,
1.0 * offers / NULLIF(interviewed, 0) AS offer_rate,
applications - LAG(applications) OVER (ORDER BY month) AS application_change
FROM monthly
ORDER BY month;

```

PORTFOLIO IDEA

Turn the final dataset into a one-page dashboard and add a README that explains the grain, data assumptions, three quality checks, and two insights. That demonstrates SQL plus professional analytical communication.

QUICK REVISION NOTES

- Design one-row-per-application sources before monthly aggregation.
- Use NULLIF to avoid division-by-zero in ratios.
- Validate distinct application counts before and after joins.
- Window functions add trend context after aggregation.
- A portfolio project should explain assumptions, not just show code.

Practice Questions

- Q1** Why aggregate interviews before joining them to applications?
- Q2** What does NULLIF protect against in rate calculations?
- Q3** What quality checks would you run before publishing the dashboard?
- Q4** What makes this project portfolio-ready?

Hands-on Exercises

- 1 Add average days to first interview.
- 2 Find the top application source per month using ROW_NUMBER.
- 3 Add a rolling three-month application average.
- 4 Write three business insights from a sample result.

Course Summary

Professional SQL is not defined by how many functions you know. It is defined by whether your queries preserve the right grain, answer the intended question, survive edge cases, and can be understood by someone else.

Query foundations

Start with the business question and the grain. SELECT, WHERE, and ORDER BY are simple tools, but they become reliable only when the row meaning is explicit.

Metrics and joins

Aggregations change grain, and joins can multiply it. Count deliberately, place filters carefully, and validate row counts around every important join.

Analytical logic

CASE, NULL handling, dates, CTEs, and window functions turn raw tables into interpretable analysis while keeping assumptions visible.

Professional performance

Optimize after correctness. Reduce unnecessary work, understand index thinking, inspect query plans, and prefer maintainable SQL over clever SQL.

Quick Revision Notes - The Whole Book

WHOLE-BOOK CHECKLIST

- Always define the grain of the final result.
- WHERE filters detail rows; HAVING filters grouped rows.
- GROUP BY changes the level of detail.
- LEFT JOIN can become effectively inner if right-side filters are placed in WHERE.
- NULL means unknown or missing, not zero.
- CTEs make intermediate reasoning testable.
- Window functions calculate across related rows without collapsing detail.
- ROW_NUMBER is a reliable tool for latest-row selection when ordering is deterministic.
- Validate joins with row counts and distinct business keys.
- Correctness first, performance second, clarity throughout.

REFERENCE

Glossary of Professional SQL Terms

Term	Meaning in plain English
Grain	What one row represents in a dataset or result.
Measure	A numeric value being aggregated, such as revenue or count.
Dimension	A descriptive field used to group or slice measures, such as month or region.
Aggregation	Combining multiple rows into summary values.
GROUP BY	Defines the level at which aggregated measures are reported.
HAVING	Filters grouped results after aggregation.
Join key	Column or set of columns used to match rows between tables.
Cardinality	The relationship between table rows, such as one-to-many.
NULL	A marker for missing or unknown data.
COALESCE	Returns the first non-NULL expression from a list.
CASE	Conditional logic that maps rows into values or categories.
CTE	A named intermediate query defined with WITH.
Window function	A calculation across related rows that preserves the underlying rows.
PARTITION BY	Defines groups inside a window calculation.
ROW_NUMBER	Assigns a sequential number within an ordered window.
LAG	Returns a value from a prior row in window order.
Index	A database structure that can provide a faster path to selected rows.
Query plan	The database engine's strategy for executing a query.
Selectivity	How narrowly a filter reduces the candidate rows.
Data quality check	A test used to validate completeness, uniqueness, ranges, or relationships before interpretation.

20-Question Professional SQL Quiz

1. What does the grain of a result describe?
2. Which clause filters rows before aggregation?
3. Which clause filters groups after aggregation?
4. What does COUNT(column) ignore?
5. What relationship can multiply rows in a join?
6. Where can a right-table condition be placed to preserve unmatched rows in a LEFT JOIN?
7. How should SQL test for missing values?
8. What does COALESCE return?
9. Why does CASE order matter?
10. What is a CTE?
11. How does a window function differ from GROUP BY?
12. What does PARTITION BY define?
13. What is ROW_NUMBER commonly used for?
14. What does LAG return?
15. Why validate COUNT(DISTINCT key) around joins?
16. Why can SELECT * be undesirable?
17. What is an index conceptually?
18. What should happen before performance optimization?
19. Why use NULLIF in a denominator?
20. What makes SQL “professional” beyond syntax?

Answer Key

1. What one row represents.
2. WHERE.
3. HAVING.
4. NULL values in that column.
5. One-to-many or many-to-many relationships.
6. In the ON clause, when that is the intended logic.
7. IS NULL / IS NOT NULL.
8. The first non-NULL expression.
9. The first matching branch is used and later branches are skipped.
10. A named intermediate query defined using WITH.
11. It calculates across related rows without collapsing them.
12. Groups within a window calculation.
13. Ranking or selecting one ordered row per entity, such as the latest record.
14. A value from a prior row in the window order.
15. To detect unintended row multiplication or entity loss.
16. It reads unnecessary columns, increases coupling, and reduces clarity.
17. A structure that can provide a faster access path to rows.
18. Validate business logic and correctness.
19. To avoid division-by-zero.
20. Correct grain, defensible logic, validation, readability, and maintainability.

SQL is a career skill when it helps you make better decisions.

Keep querying. Keep validating. Keep explaining.

CareerSense Academy
careersenseai.com/resources